Project Report: Implement Client Script & UI Policy (Incident)

1. INTRODUCTION

1.1 Project Overview

This project implements client-side data controls on the ServiceNow Incident form using UI Policies, UI Policy Actions and Client Scripts. The controls make fields mandatory, show or hide fields, calculate Priority, and block submission when required conditions are not met, so that data is validated in the browser before it reaches the server.

1.2 Purpose

The purpose is to demonstrate how ServiceNow client-side controls enforce data integrity on Incident records, supporting accurate triage, routing and resolution, reliable reporting and SLA compliance.

2. IDEATION PHASE

2.1 Problem Statement

Incident records require consistent and accurate data entry to ensure effective triage, routing and resolution. Relying solely on user awareness and manual checks leads to incomplete, inconsistent or incorrect data being submitted. This affects reporting accuracy, SLA compliance and overall service quality.

Statement Element	Description
Who is affected	Service desk agents, fulfillers, incident managers and reporting consumers
Problem	Incomplete or inconsistent Incident data is saved without validation
Impact	Mis-routed tickets, inaccurate Priority, missing resolution evidence, unreliable reports and SLA breaches

2.2 Empathy Map Canvas

Persona	Says	Thinks	Does	Feels
Service Desk Agent	"I need to log tickets quickly."	Optional fields can be skipped	Submits with minimal details	Pressured by volume
Resolver Group Member	"This ticket has no useful details."	Routing and diagnosis are delayed	Returns tickets for clarification	Frustrated by rework
Incident Manager	"The priority values do not match the impact."	Reports and SLA data are unreliable	Audits and corrects records manually	Concerned about compliance

2.3 Brainstorming

- Make Subcategory mandatory when Category is Software, and expose the Assignment Group field.
- Make Work notes mandatory for Priority 1 - Critical incidents.
- Calculate Priority automatically from the Impact and Urgency matrix.
- Abort submission when Short description is under 10 characters or Work notes are empty on Resolve; require Resolution details at Resolve.
- Selected approach: UI Policies for field behavior, Client Scripts for calculation and validation.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

Stage	User Action	Control Applied	Outcome
Create	Opens a new Incident form	onChange scripts calculate Priority	Priority is consistent with Impact and Urgency
Categorize	Selects Category	UI Policy adjusts Subcategory and Assignment Group	Correct routing data captured
Escalate	Sets Priority to 1 - Critical	UI Policy makes Work notes mandatory	Escalation is documented
Submit	Saves the record	onSubmit script validates input	Invalid records are blocked
Resolve	Sets State to Resolved	UI Policy and onSubmit script enforce resolution data	Complete resolution evidence is recorded

3.2 Solution Requirement

Functional requirements

ID	Requirement
FR-01	Category = Software shall make Subcategory mandatory and show Assignment Group.
FR-02	Priority = 1 - Critical shall make Work notes mandatory.
FR-03	State = Resolved shall make Resolution code and Resolution notes mandatory.
FR-04	State = Closed shall make Short description and Caller read-only.
FR-05	Priority shall be auto-populated when Impact or Urgency changes.
FR-06	Subcategory shall be cleared when Category changes.
FR-07	Submission shall be aborted when Short description has fewer than 10 characters.
FR-08	Submission shall be aborted when State is Resolved and Work notes are empty.
FR-09	Each validation failure shall display a clear message to the user.

Non-functional requirements

- Controls execute on the client and shall not require server round trips.
- Configuration shall be applied to the Incident [incident] table only and shall use standard platform APIs.
- Controls shall not alter the behavior of unrelated forms.

3.3 Data Flow Diagram

Step	Source	Process	Destination
1	User	Enters or changes field values on the Incident form	Form (browser)
2	Form	UI Policies evaluate conditions and apply actions (mandatory, visible, read-only)	Form fields
3	Form	onChange Client Scripts read Impact, Urgency and Category and set Priority or Subcategory	Form fields
4	User	Clicks Save, Submit or Update	onSubmit Client Script
5	onSubmit Client Script	Validates Short description length and Work notes; returns false on failure	Message shown to user
6	onSubmit Client Script	Returns true when all checks pass	Incident table (server)

3.4 Technology Stack

Component	Technology
Platform	ServiceNow
Application	Incident Management (incident table)
Configuration	UI Policy and UI Policy Actions (System UI)
Scripting	Client Scripts (JavaScript) using g_form and g_user APIs
Client	Web browser with the Core UI form

4. PROJECT DESIGN

4.1 Problem Solution Fit

Problem	Solution
Missing categorization details	UI Policy makes Subcategory mandatory for Software incidents
Critical incidents lack documentation	UI Policy makes Work notes mandatory for Priority 1 - Critical
Priority inconsistent with Impact and Urgency	onChange Client Scripts apply the Priority matrix
Poor-quality descriptions and undocumented resolutions	onSubmit Client Script aborts invalid submissions

4.2 Proposed Solution

The solution combines declarative UI Policies for conditional field behavior with Client Scripts for calculation and validation logic. All records and scripts apply to the Incident table.

UI Policies

No.	Name	Condition	Actions
UP1	Software Category Requirements	Category is Software	Subcategory: Mandatory = True, Visible = True. Assignment Group: Visible = True.
UP2	Critical Priority Work Notes	Priority is 1 - Critical	Work notes: Mandatory = True.
UP3	Resolution Details Required	State is Resolved	Resolution code: Mandatory = True. Resolution notes: Mandatory = True.
UP4	Lock Closed Incident	State is Closed	Short description: Read only = True. Caller: Read only = True.

Each UI Policy has Reverse if false and On load enabled, so that field behavior resets when the condition no longer applies.

Client Scripts

Name	Type	Field	Logic
Set Priority on Impact	onChange	Impact	Read Impact and Urgency. When both are set, look up the matrix value and call g_form.setValue('priority', value).
Set Priority on Urgency	onChange	Urgency	Same matrix logic as above, triggered by an Urgency change.
Reset Subcategory	onChange	Category	When Category changes after load, clear Subcategory with g_form.clearValue('subcategory').
Validate Incident Submit	onSubmit	Short description, Work notes	Abort when Short description length is under 10 characters. Abort when State is Resolved and Work notes are empty. Show message with g_form.addErrorMessage() and return false.

Priority matrix (Impact x Urgency)

Impact \ Urgency	1 - High	2 - Medium	3 - Low
1 - High	1 - Critical	2 - High	3 - Moderate
2 - Medium	2 - High	3 - Moderate	4 - Low
3 - Low	3 - Moderate	4 - Low	5 - Planning

4.3 Solution Architecture

Browser-side execution order on the Incident form: (1) form loads and UI Policies with On load enabled apply; (2) field changes trigger UI Policies and onChange Client Scripts; (3) on Save or Submit the onSubmit Client Script runs; (4) a true return sends the record to the server, and a false return halts submission and keeps the user on the form.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Phase	Activity	Deliverable
1	Requirement analysis and design of validation rules	Requirement and design documents
2	Configure UI Policies and UI Policy Actions	UP1 to UP4
3	Develop Client Scripts	Two onChange scripts, one Category reset script, one onSubmit script
4	Execute test cases and fix defects	Test results
5	Final documentation	Project report

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The following test cases verify each requirement on the Incident form.

ID	Scenario	Test Steps	Expected Result
TC-01	Software category rule	Set Category = Software	Subcategory is visible and mandatory; Assignment Group is visible
TC-02	Non-software category	Change Category to Hardware	Subcategory is no longer mandatory (UI Policy reversed)
TC-03	Critical priority rule	Set Priority = 1 - Critical	Work notes become mandatory; save without notes is blocked
TC-04	Non-critical priority	Set Priority = 3 - Moderate	Work notes are not mandatory
TC-05	Priority auto-populate	Set Impact = 1, Urgency = 1	Priority is set to 1 - Critical
TC-06	Priority matrix (mid)	Set Impact = 2, Urgency = 3	Priority is set to 4 - Low
TC-07	Priority matrix (low)	Set Impact = 3, Urgency = 3	Priority is set to 5 - Planning
TC-08	Category reset	Select a Subcategory, then change Category	Subcategory is cleared
TC-09	Short description too short	Enter 5 characters and Save	Submission aborted; error message shown
TC-10	Short description valid	Enter 10 or more characters and Save	Record is saved
TC-11	Resolve without notes	Set State = Resolved, leave Work notes empty, Save	Submission aborted; error message shown
TC-12	Resolve without resolution data	Set State = Resolved, leave Resolution code and notes empty	Fields are flagged mandatory; save is blocked
TC-13	Resolve with complete data	Provide Work notes, Resolution code and Resolution notes	Record is saved as Resolved
TC-14	Closed record lock	Open an incident in State = Closed	Short description and Caller are read-only

Performance criteria. All controls run in the browser and add no server calls to form interaction. The form shall respond to field changes without a visible delay, and the form shall load without added client errors in the browser console.

7. RESULTS

7.1 Output Screenshots

The expected on-screen outcomes for verification are listed below. Screenshots are captured from the ServiceNow instance during test execution.

Reference	Screen to Capture	Expected Display
Figure 1	Incident form with Category = Software	Subcategory marked mandatory; Assignment Group displayed
Figure 2	Incident form with Priority = 1 - Critical	Work notes marked mandatory
Figure 3	Impact and Urgency selected	Priority populated automatically
Figure 4	Submit with short Short description	Error message and submission halted
Figure 5	Resolve with empty Work notes	Error message and submission halted

8. ADVANTAGES & DISADVANTAGES

Advantages

- Immediate feedback to the user without a server round trip.
- Improves data completeness and consistency for triage and routing.
- Improves Priority accuracy and reporting reliability.
- Supports SLA compliance through complete resolution data.
- Uses standard, configurable platform features.

Disadvantages

- Client-side controls do not apply to imports, integrations or API updates, so server-side Business Rules or Data Policies are needed for full enforcement.
- Client Scripts can be bypassed when the form is not used.
- Too many scripts on a form can slow form load and increase maintenance effort.

9. CONCLUSION

The project demonstrates that UI Policies and Client Scripts enforce data integrity on Incident records at the point of entry. Conditional mandatory and visible fields, automatic Priority calculation and submit-time validation reduce incomplete and inconsistent data, which improves triage, routing, reporting accuracy and SLA compliance. The skills demonstrated are Incident Management, UI Policy, UI Policy Actions, Client Scripts and Form Validation.

10. FUTURE SCOPE

- Add server-side Business Rules or Data Policies to enforce the same rules for non-UI updates.
- Extend the controls to the Service Catalog and Change Management forms.
- Replace the hard-coded Priority matrix with a Data Lookup definition.
- Add Catalog UI controls and role-based exemptions for specific user groups.

11. APPENDIX

Source Code (onSubmit Client Script: Validate Incident Submit)

```
function onSubmit() {
  var shortDesc = g_form.getValue('short_description') || '';
  var state = g_form.getValue('state');
  var workNotes = g_form.getValue('work_notes') || '';

  if (shortDesc.trim().length < 10) {
    g_form.addErrorMessage('Short description must be at least 10 characters.');
    return false;
  }
  if (state == '6' && workNotes.trim() === '') {
    g_form.addErrorMessage('Work notes are required to resolve an incident.');
    return false;
  }
  return true;
}
```

Source Code (onChange Client Script: Set Priority on Impact; same logic on Urgency)

```
function onChange(control, oldValue, newValue, isLoading) {
  if (isLoading || newValue === '') return;
  var impact = g_form.getValue('impact');
  var urgency = g_form.getValue('urgency');
  if (!impact || !urgency) return;
  var matrix = {
    '1': {'1': '1', '2': '2', '3': '3'},
    '2': {'1': '2', '2': '3', '3': '4'},
    '3': {'1': '3', '2': '4', '3': '5'}
  };
  g_form.setValue('priority', matrix[impact][urgency]);
}
```

Dataset Link: Not applicable. No external dataset is used.

GitHub & Project Demo Link: Not applicable. The configuration resides in the ServiceNow instance.